use.ai

Shopify Storefront API Query

User:

[File: pasted-text.txt] [File: pasted-text.txt] [File: pasted-text.txt]

I am building kitchen planner on cursor with no coding experience and require a simple, clean app that is user friend, and used over all devices like laptop, ipad, iphone ios and android. The kitchen plan ner need to be integrated with shopify ecommerce website, using the ecommerce catalogue and SKU to show total price in live. the user will have ecommerce account to access this kitchen planner, I still have some issues with wall drawing user experience to solve, making the layout to fit mobile android, ios and computer. Ideally the user can use one of the bluetooth measure device to insert measurement. the mvp now is to make all the wall design and installing the 3D glb model cabinet seemlessly via connection on ecommerce website. please also suggest best way to store save and store users login and project information. I need to have prototype very soon, like in 1-2days, so i'll need to be able to pass a link to 3rd party for testing and comment on experience. Future phases after shoppify launch will be realistic render photos, gaming like views like first eye view with mobile game controller. save and download the quotation with all the sku info with the photos and design in plan view and elevation view showing overal and each cabinets dimension..etc more function in 3months time and debug for white label and sell subscription to third party enterprice client. please give me a full recommendation on architecture, next step and next phases. tell me how much you can help before with higheshest efficiency before i use sonnet 4.6 back as routine.

Assistant:

You've actually built a *lot* already — what you have is genuinely impressive for someone with no coding experience. Let me give you a clear-eyed plan that gets you to a testable link in 1-2 days and a Shopify-integrated MVP shortly after, then phases out the bigger features.

Where you actually are

Your `main.js` is a working browser-based 3D/2D planner with: wall drawing (multiple modes), GLB cabinet import, elevation view, undo/redo, touch + desktop input, room area, basic quote totals, CSV export. The hard part — the 3D engine and interaction — is done. What's missing is the **plumbing around it**: hosting, accounts, persistence, Shopify catalog, and UX polish for mobile.

Don't rebuild any of it. Wrap it.

Recommended architecture (cheap, fast, prototype-friendly)

Layer	Pick	Why
Hosting (frontend)	**Vercel** or **Netlify** (free tier)	Drag-and-drop deploy from GitHub. Live URL in 5 minutes. HTTPS by default (required for Bluetooth + Shopify embed).
Auth + database	**Supabase** (free tier)	Email/Google login, Postgres database, file storage, all in one. Has a JS SDK you paste in. No backend code needed for MVP.
Shopify integration	**Shopify Storefront API** (read-only, public token)	Pulls product catalog + live prices + SKUs into your planner via JavaScript. No server needed. Customer accounts can be linked later via Shopify Customer Account API.
File storage (GLB, screenshots, PDFs)	Supabase Storage	Same account as auth.
PDF/quote export	`jspdf` + `html2canvas` (later)	Browser-side, no server.
Code hosting	GitHub (free)	Vercel auto-deploys on every push. Cursor commits directly.

Total monthly cost for prototype + early users: $0. You only start paying when you cross Supabase's free tier (~50k monthly users) or Vercel's bandwidth.

The 1-2 day path to a testable link

Day 1 (4-6 hours)

Push your existing code to GitHub. Cursor has a one-click "publish to GitHub" in the source control panel.

`your-planner.vercel.app`. Share this with testers immediately — it works as is.

Add a Supabase project. Copy two env variables (`SUPABASE_URL`, `SUPABASE_ANON_KEY`) into Vercel. Add their auth widget — ~30 lines of code Cursor can write for you. Done: email/Google login.

Add a "Save Project" button. Serialise your `walls`, `placedItems`, and `settings` to JSON, save to a Supabase `projects` table keyed by `user_id`. Add a "My Projects" list. This is ~50 lines.

That gets you: live URL + login + save/load. Enough for third-party testers.

Day 2 (4-6 hours)

Replace your hard-coded `products` array with a Shopify Storefront API call. One `fetch()` to your store's `/api/2024-10/graphql.json` endpoint pulls all products, SKUs, prices, and metafields (where you'll store `modelPath`, `width`, `height`, `depth`). Map the response into your existing product format — your planner doesn't need to change.

Polish mobile UX. The two biggest pain points testers will hit:

Toolbar is cramped on iPhone — collapse less-used buttons into a "⋯" menu.

Draw-wall mode menu sometimes overlaps the confirm bar — give them mutually exclusive z-index zones.

Deploy. Share link.

Wall drawing UX — the issues I can see in your code

A few specific things worth fixing before testers see it:

Quick Draw forces 90° only. Your `mousemove` handler hard-locks every wall to horizontal/vertical from `wallStart`. That's fine as a default but frustrating for diagonal walls. Consider: free angle, but snap to 90° when within ~5° tolerance (you already have `snapTo90` — just invert when it applies).

Glide draw is clever but invisible until you try it. First-time users won't discover it. Add a one-time tooltip or a small "🖍️ tip: hold and drag to trace" hint the first time someone opens the draw menu.

Wall handles are tiny on iPad (0.15 radius). Bump to 0.22 for touch, keep 0.15 for mouse. Detect via `IS_TOUCH`.

`wallDimLocked = true`, change the preview line colour to green so the user knows the next click will commit at that exact length.

Closing a room is fragile — the 0.2m proximity to `firstPoint` is generous on desktop but easy to miss on mobile. Increase to 0.4m on touch.

None of these are blockers for a prototype link. Day 3-4 polish.

Bluetooth measure devices

This is more realistic than people think. The **Web Bluetooth API** works in Chrome on Android/Windows/Mac/Linux (not iOS Safari — Apple still blocks it). Most popular laser distance tools (Bosch GLM 50C, Leica Disto D2/D110, Stabila LD250BT) expose their readings over BLE with documented GATT services.

MVP approach: Build for one device first (Leica Disto has the best-documented protocol and a JS library exists). On iOS, fall back to manual entry — your `BT` buttons already do this. Tag this as **Phase 2**, not MVP.

Phasing — what to do when

Phase 0 — this week (prototype link)

Vercel + Supabase + login + save/load + Shopify catalog fetch. Share with 3-5 testers.

Phase 1 — weeks 2-4 (Shopify MVP launch)

Embed planner inside your Shopify store as a custom page (just an `<iframe>` pointing to your Vercel URL, or a Shopify App Block).

Link Shopify customer accounts to Supabase users (Shopify sends a signed JWT you can verify).

"Add all to cart" button — pushes selected SKUs into the Shopify cart via their AJAX cart API.

PDF quote export with floor plan screenshot + elevation thumbnails + line-item table.

Polished mobile UX.

Phase 2 — months 2-3

Bluetooth measure device support (Android/desktop only).

Realistic rendering (swap MeshBasicMaterial for MeshPhysicalMaterial + an HDRI environment map — ~1 day of work in three.js).

First-person walkthrough (you already have `camera3D` — add WASD/touch joystick controls and a `pointerlock` mode).

Gamepad support via the Web Gamepad API (built into browsers, no plugins).

Project sharing links (read-only public URLs).

Phase 3 — months 4-6

White-label theming (CSS variables driven by a tenant config in Supabase).

Multi-tenant subscriptions via Stripe Billing.

Admin dashboard for enterprise clients to manage their catalog.

Versioned project history.

Data model for Supabase (paste this into Cursor)

```
users                    -- Supabase auth handles this
projects
  id, user_id, name, created_at, updated_at, thumbnail_url
  scene_json (jsonb)     -- your walls + placedItems + settings
  shopify_cart_token     -- nullable, for resuming carts
catalogs                 -- optional override of Shopify data
  tenant_id, shopify_product_id, glb_url, width_mm, height_mm, depth_mm
tenants                  -- for Phase 3 white-label
  id, name, shopify_store_domain, theme_json
```

`scene_json` is the magic: serialise your whole planner state to one JSON blob. Don't try to normalise walls and items into separate tables yet — that's a Phase 3 problem.

How much I can help right now vs. switching back to Sonnet

Honest answer: **I'm best for the architecture, planning, and tricky three.js debugging** you've already seen me do here. For the *volume* of small refactors and CSS tweaks you'll burn through in the next two days, Sonnet 4.6 is faster and cheaper per token, and the work isn't subtle enough to need me.

Use me for:

- Designing the Supabase schema and the Shopify GraphQL query (1 message).
- Writing the auth + save/load module (1-2 messages).
- The Bluetooth Disto integration when you get there (the GATT protocol is fiddly).
- Any time three.js does something weird and you can't figure out why.
- Reviewing the final architecture before you commit to it.

Use Sonnet 4.6 for:

- All the day-to-day "make this button bigger on iPhone" work.
- CSS, layout, copy changes.
- Wiring up the Shopify cart button.
- Generating product seed data.

A good pattern: draft the plan with me, hand the plan to Sonnet inside Cursor for execution, come back to me when something doesn't work or a decision needs making.

One thing to do before you write any more code

Decide your **Shopify store URL** and **whether the planner lives at `planner.yourstore.com` or inside the Shopify storefront as an embedded page**. This affects how auth flows. Embedded is friendlier for customers (single login) but requires Shopify App Bridge. A separate subdomain is simpler to ship in 2 days. For the prototype, go separate subdomain; migrate to embedded in Phase 1.

If you want, share your Shopify store URL and I'll write the exact Storefront API query you'll need to pull your cabinet catalog with GLB metafields — that's the one piece that'll unblock Day 2.

User:

brownboxkit.co.nz

Assistant:

Let me take a look at your store so the Shopify integration advice is grounded in what you actually have.

Good — I can see your store clearly now. A few useful facts before the recommendations:

- You're already on **standard Shopify** (the `/products.json` endpoint works), which means the Storefront API will work out of the box.
- Catalog is structured as you'd hope: `product_type` (Base Cabinet, Wall Cabinet), clean SKU codes like `B3-800-CRNR` and `L2-2200`, prices in NZD, `compare_at_price` for sale logic, weights in grams for shipping. All of this is fetchable from the planner with a single API call.
- Your homepage already advertises *"simply drag & drop — Introducing our latest 3D kitchen designer"*, so the planner has a marketing slot waiting for it.
- I noticed several products tagged "(Draft)" — worth filtering those out of the planner catalog so testers don't see WIP items.

What this means for your build

1. SKU naming is your friend

Your SKU pattern (`B3-800-CRNR`, `L2-2200`) encodes the dimensions. You can parse the width directly from the SKU code as a fallback if a product is missing metafields. That's a Phase 1 robustness win.

2. You need metafields, not a separate database

For each product in Shopify Admin, add these **product metafields** under a custom namespace (e.g. `planner`):

Metafield	Type	Example	Purpose
`planner.glb_url`	URL	`https://cdn.shopify.com/.../b3-800-crnr.glb`	3D model file
`planner.width_mm`	Integer	`800`	Override / canonical width
`planner.height_mm`	Integer	`720`	Cabinet height
`planner.depth_mm`	Integer	`560`	Cabinet depth
`planner.category`	Single-line text	`base` / `wall` / `tall` / `corner`	Snapping & placement rules
`planner.thumbnail_url`	URL	(optional)	Custom catalog icon

Upload your GLB files to **Shopify Files** (Settings → Files) — Shopify gives you a permanent CDN URL, no separate hosting needed. This keeps everything inside one system your team already manages.

3. The exact query you'll need for Day 2

Replace your hard-coded `products` array with this. Cursor can wire it up in 10 minutes.

```
 // At the top of main.js, before the products array
const SHOPIFY_DOMAIN = 'brownboxkit.co.nz';
const SHOPIFY_TOKEN  = 'YOUR_STOREFRONT_ACCESS_TOKEN'; // public, safe in frontend

async function fetchShopifyCatalog() {
  const query = `{
    products(first: 100, query: "product_type:'Base Cabinet' OR product_type:'Wall Cabinet'") {
      edges {
        node {
```

```
          id
          title
          handle
          productType
          featuredImage { url }
          variants(first: 1) {
            edges {
              node {
                id
                sku
                price { amount currencyCode }
                compareAtPrice { amount }
              }
            }
          }
          glbUrl:        metafield(namespace: "planner", key:
"glb_url")        { value }
          widthMm:       metafield(namespace: "planner", key:
"width_mm")       { value }
          heightMm:      metafield(namespace: "planner", key:
"height_mm")      { value }
          depthMm:       metafield(namespace: "planner", key:
"depth_mm")       { value }
          category:      metafield(namespace: "planner", key:
"category")       { value }
        }
      }
    }
  }`;

  const res = await fetch(`https://${SHOPIFY_DOMAIN}/api/2024-
10/graphql.json`, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'X-Shopify-Storefront-Access-Token': SHOPIFY_TOKEN
    },
    body: JSON.stringify({ query })
  });

  const json = await res.json();
```

```
  const json = await res.json();
  return json.data.products.edges
    .filter(({ node }) => !node.title.includes('(Draft)')) // skip WIP
    .map(({ node }) => ({
      id:        node.handle,
      name:      node.title,
      sku:       node.variants.edges[0]?.node.sku,
      variantId: node.variants.edges[0]?.node.id,    // needed for add-
to-cart
      modelPath: node.glbUrl?.value || null,
      width:     parseInt(node.widthMm?.value)  || 600,
      height:    parseInt(node.heightMm?.value) || 720,
      depth:     parseInt(node.depthMm?.value)  || 560,
      thumbnail: node.featuredImage?.url,
      skus: [{
        id:    node.variants.edges[0]?.node.sku,
        label: node.productType,
        price: parseFloat(node.variants.edges[0]?.node.price.amount)
      }]
    }));
}

// Replace your hard-coded products section with:
fetchShopifyCatalog().then(catalog => {
  products.push(...catalog);
  catalog.forEach(product => {
    const div = document.createElement('div');
    div.className = 'product-item';
    div.textContent = product.name;
    div.addEventListener('click', () => placeProduct(product));
    productList.appendChild(div);
  });
});
```

4. Getting the Storefront Access Token (5 minutes)

Shopify Admin → **Settings** → **Apps and sales channels** → **Develop apps** → **Create an app**

Shopify Admin → **Settings** → **Apps and sales channels** → **Develop apps** → **Create an app** → name it "Kitchen Planner" → **Configure Storefront API scopes** → tick `unauthenticated_read_product_listings`, `unauthenticated_read_product_inventory`, `unauthenticated_write_checkouts` (for cart later) → Install → copy the **Storefront access token**. It's safe to embed in frontend code — that's what it's designed for.

5. "Add to Cart" — the line that closes the loop

When the customer is done designing, this single function moves every placed item into their Shopify cart:

```
async function sendToShopifyCart() {
  const lineItems = placedItems
    .filter(m => m.userData?.product?.variantId)
    .map(m => ({
      merchandiseId: m.userData.product.variantId,
      quantity: 1
    }));

  const mutation = `mutation cartCreate($input: CartInput!) {
    cartCreate(input: $input) {
      cart { checkoutUrl }
    }
  }`;

  const res = await fetch(`https://${SHOPIFY_DOMAIN}/api/2024-
10/graphql.json`, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'X-Shopify-Storefront-Access-Token': SHOPIFY_TOKEN
    },
    body: JSON.stringify({
      query: mutation,
      variables: { input: { lines: lineItems } }
    })
  });

  const { data } = await res.json();
  window.location.href = data.cartCreate.cart.checkoutUrl;
}
```

That redirects the customer to your Shopify checkout with everything pre-loaded. The whole planner becomes a sales funnel.

Revised 2-day plan, now Brown Box-specific

Today (Day 1)

- Push code to GitHub → deploy to Vercel → live URL

Push code to GitHub → deploy to Vercel → live URL.

Create Shopify private app, get Storefront token, paste into a Vercel environment variable.

Wire up `fetchShopifyCatalog()` — replace the hard-coded products array.

Upload 2-3 GLB files to Shopify Files, add metafields to 2-3 cabinets so you have real test data.

Set up Supabase project, add Google login + Save Project button.

Tomorrow (Day 2)

Add "Send to Cart" button → redirects to your Shopify checkout.

Filter out `(Draft)` products and any without GLB metafields (or show them as placeholder boxes).

Mobile UX pass — collapse toolbar on iPhone, enlarge wall handles on touch.

Test on iPhone, iPad, Android, desktop. Share link with testers.

You'll be sharing `planner.brownboxkit.co.nz` (point a subdomain at Vercel) with testers by Friday NZ time.

Two things to decide before tomorrow

Which 3-5 cabinets do you have GLB models for already? Start metafield setup with those. The planner can fall back to grey boxes for the rest while you build out the model library.

Subdomain vs Shopify-embedded? For testing, point `planner.brownboxkit.co.nz` at Vercel (5-minute DNS change in Shopify Domains). For Phase 1 launch, embed it as a Shopify page with `<iframe src="https://planner.brownboxkit.co.nz">`. Same code, two access points.

Where I'd spend a Claude message vs Sonnet

The pattern I'd suggest: come back to me when you hit one of these moments, otherwise let Sonnet drive in Cursor.

Worth a message to me: Supabase schema sign-off, the Shopify customer-account → Supabase JWT bridge in Phase 1, designing the metafield structure for tall/corner/island cabinets (geometry gets tricky), the PDF quote layout in Phase 1, Bluetooth Disto GATT integration in Phase 2.

Hand to Sonnet: all the metafield data entry helper scripts, CSS tweaks, the cart button styling, mobile toolbar collapse, GLB file naming conventions, test data.

When you're ready to start, the very first thing to do is the Shopify Files upload and metafield setup

When you're ready to start, the very first thing to do is the Shopify Files upload and metafield setup for one cabinet — say `B3-800-CRNR`. That gives you the proof that the pipeline works end-to-end before you scale it to your full catalog.

User:

ok. moving on. send me full list of scope with phases clealy divide the scopes between opus and sonnet. then for the sonnet also write me the opening instruction i can just copy and paste with the latest html, main.js file when i use sonnet. Go.

Assistant:

Brown Box Kit Planner — Full Scope & Workflow

Phase 0 — Live Prototype Link (Day 1-2)

Goal: A working URL you can send to testers tomorrow. Login, save, real Shopify catalog, mobile-usable.

#	Task	Owner
0.1	Push existing repo to GitHub, connect to Vercel, get live URL	**Sonnet**
0.2	Buy/point `planner.brownboxkit.co.nz` subdomain at Vercel	**Sonnet** (steps)
0.3	Create Shopify private app, generate Storefront API token	**Sonnet** (steps)
0.4	Replace hard-coded `products` array with Shopify GraphQL fetch	**Sonnet**
0.5	Filter out `(Draft)` products, fall back to grey boxes when GLB missing	**Sonnet**
0.6	Create Supabase project, add `projects` table, paste auth widget	**Sonnet**
0.7	"Save Project" + "My Projects" list (serialise walls + items to JSON)	**Sonnet**
0.8	"Send to Shopify Cart" button → checkout redirect	**Sonnet**
0.9	Mobile toolbar collapse (iPhone), larger wall handles on touch	**Sonnet**
0.10	Upload 3-5 GLB files to Shopify Files, add metafields for those cabinets	**You (manual)**
0.11	**Schema sign-off + Shopify→Supabase data flow design**	**Opus**
0.12	**Debug any three.js / GLB scaling weirdness during testing**	**Opus**

Phase 1 — Shopify MVP Launch (Week 2-4)

Goal: Embedded in your storefront, real customers using it, quote-to-cart loop closed.

#	Task	Owner
1.1	Embed planner as Shopify page (`<iframe>` or App Block)	**Sonnet**
1.2	Link Shopify Customer Accounts → Supabase user via signed JWT	**Opus** (tricky auth bridge)
1.3	PDF quote export — floor plan + elevations + SKU line items + dimensions	**Opus** (layout design) → **Sonnet** (implementation)
1.4	Polish all wall-drawing UX issues (90° snap, glide hint, close-room tolerance)	**Sonnet**
1.5	Loading states, error toasts, empty-cart guards	**Sonnet**
1.6	Project thumbnails (screenshot canvas → Supabase Storage)	**Sonnet**
1.7	Catalog admin: helper script to bulk-edit metafields from CSV	**Sonnet**
1.8	Analytics — track wall draws, products placed, carts sent	**Sonnet**
1.9	Read-only shareable project links (`/p/abc123`)	**Sonnet**
1.10	Tall/corner/island cabinet geometry rules (snap-to-corner, height tiers)	**Opus**
1.11	Privacy policy, terms, GDPR-style data export button	**Sonnet**
1.12	**Architecture review before launch**	**Opus**

Phase 2 — Pro Features (Month 2-3)

Goal: Differentiate from competitors. Make the planner feel premium.

#	Task	Owner
2.1	Realistic rendering (MeshPhysicalMaterial + HDRI + soft shadows)	**Opus** (material setup) → **Sonnet** (UI toggle)
2.2	First-person walkthrough mode (WASD + pointer lock + mobile joystick)	**Opus** (camera math) → **Sonnet** (controls UI)
2.3	Gamepad support via Web Gamepad API	**Sonnet**
2.4	Bluetooth Disto / Bosch GLM measure device integration	**Opus** (GATT protocol is fiddly)
2.5	Manual fallback: type measurement → BT button on iOS	**Sonnet**
2.6	Material/colour swatches per cabinet (laminate options)	**Sonnet**
2.7	Benchtop generation (auto-extrude across base cabinet tops)	**Opus**
2.8	Splashback & appliance placeholder library	**Sonnet**
2.9	Offline mode (service worker, cached catalog)	**Sonnet**
2.10	iPad Pro Apple Pencil precision drawing	**Sonnet**

Phase 3 — White-Label & Subscriptions (Month 4-6)

Goal: Sell the planner to other cabinet retailers as a SaaS.

#	Task	Owner
3.1	Multi-tenant data model (`tenants` table, scoped RLS policies)	**Opus**
3.2	Tenant-specific theming via CSS variables	**Sonnet**
3.3	Admin dashboard for tenants to manage their catalog	**Sonnet**
3.4	Stripe Billing integration (subscription tiers)	**Opus** (webhook flow) → **Sonnet** (UI)
3.5	Per-tenant Shopify store connection wizard	**Opus**
3.6	Usage metering (designs created, models uploaded)	**Sonnet**
3.7	Versioned project history + restore	**Sonnet**
3.8	Team accounts (designers shared across one tenant)	**Opus**
3.9	Public API for tenants to integrate with their own systems	**Opus**
3.10	White-label marketing site + onboarding flow	**Sonnet**

Division of Labour — The Rule

Opus when: architecture choices, auth/security flows, three.js math, fiddly protocols (Bluetooth, JWT), schema design, debugging when "it should work but doesn't", reviews before any launch.

Sonnet when: writing code from a spec, CSS, copy, refactors, wiring known APIs, data entry helpers, polish.

A useful test: *if a wrong answer would cost you a day of rework, ask Opus first.* Otherwise Sonnet is faster and cheaper.

Sonnet Opening Prompt — copy/paste this

When you start a Sonnet session in Cursor, paste this block at the top of your first message, then attach `main.js` and `index.html` (and `style.css`):

```
You are helping me ship a Shopify-integrated 3D kitchen planner for
Brown Box Kit
```

(brownboxkit.co.nz). The full architecture has already been designed by Opus —
your job is to implement tasks I give you, cleanly and without re-architecting.

PROJECT CONTEXT
- Brown Box Kit is a New Zealand flat-pack kitchen cabinet retailer on Shopify.
- The planner is browser-based, built with three.js, vanilla JS, no framework.
- It supports desktop, iPad, iPhone, and Android.
- Catalog comes from Shopify Storefront API (not hard-coded).
- Auth + project storage uses Supabase (free tier).
- Hosting is Vercel, deployed from GitHub.
- Pricing is NZD. Customer is in NZ.

WHAT'S ATTACHED
- main.js — the full planner: walls, GLB import, elevation view, undo/redo,
  touch + desktop input, quote totals, CSV export. ~2000 lines, working.
- index.html — toolbar, panels, modals, drawer.
- style.css — responsive layout, iPhone/iPad breakpoints already in place.

ARCHITECTURE DECISIONS — DO NOT CHANGE WITHOUT ASKING
- Scene state lives in `walls` and `placedItems` arrays. Serialise these as
  JSON to save projects.
- Shopify Storefront API access token is public-safe — embed in frontend.
- Product metafields live in `planner.*` namespace: glb_url, width_mm,
  height_mm, depth_mm, category.
- Save Project → Supabase `projects` table, `scene_json` jsonb column,
  keyed by `user_id`.
- "Send to Cart" uses Shopify cartCreate mutation, redirects to
  cart.checkoutUrl.
- Wall handles must be ~0.22 radius on touch, 0.15 on mouse.
- IS_TOUCH constant already exists in main.js — use it.

HOUSE RULES FOR THIS CODEBASE

```
- Do not refactor working code. Add to it.
- Keep existing function names. Other parts of the file depend on them.
- Touch and mouse paths must both keep working. Test mentally on
iPhone,
  iPad, and desktop before declaring done.
- All measurements internally in metres; user-facing in mm.
- `mm(v)` helper converts mm → metres. Use it.
- Never break undo/redo. New actions must push history entries.
- Three.js dispose pattern is critical for GLB — geometry, materials,
  textures must all be disposed when models are swapped or removed.
  See existing `disposeModel()` for the pattern.
- CSS: respect existing breakpoints (430px iPhone, 768px tablet).
  Don't introduce new ones without reason.

WHEN YOU'RE UNSURE
- Ask before changing scene-state shape, history entry shape, or any
  function signature.
- If a request seems to require a backend, say so — we're frontend-only
  except for Supabase SDK calls.
- If a task feels like architecture (auth flow, schema, security,
three.js
  math), tell me to take it to Opus first.

OUTPUT STYLE
- Lead with the code change. Brief explanation after.
- For multi-file changes, show each file's diff separately.
- No long preambles. I'll read the code.

The first task is below.
```

Drop that block, then add your actual task underneath. Example for your first Sonnet session:

> ***Task 1:*** *Add a* `fetchShopifyCatalog()` *function that replaces the hard-coded products array. Use the Storefront API at* `brownboxkit.co.nz`*. Filter out products with "(Draft)" in the title. Token will be in* `window.SHOPIFY_TOKEN` *(I'll inject it via Vercel env). Map response into the existing product shape so* `placeProduct()` *and* `loadProductModel()` *keep working unchanged.*

When Sonnet finishes that, your next task is "0.6 — Supabase Save Project," and so on down the Phase 0 list.

Come back to me for **0.11, 0.12, 1.2, 1.10, 1.12, 2.1, 2.2, 2.4, 2.7, 3.1, 3.4, 3.5, 3.8, 3.9.** Those are the ones where a wrong answer costs you days.